



Voglio creare la UI per un firewall, dovrei utilizzare rest api o web socket? Ci sono altre alternative?

Ottima domanda con risvolti architetturali interessanti, soprattutto per chi gestisce infrastrutture. La risposta breve è: **usa un approccio ibrido — REST API per le operazioni CRUD e WebSocket (o SSE) per il monitoring in tempo reale**. Ecco perché e le alternative.

REST API vs WebSocket per una Firewall UI

Per una UI di un firewall, i due protocolli servono scopi distinti e non si escludono a vicenda. ^[1]

Aspetto	REST API	WebSocket
Modello	Request/Response stateless	Full-duplex, connessione persistente
Uso ideale	Configurazione regole, CRUD, autenticazione	Log in real-time, alert attivi, traffico live
Scalabilità	Orizzontale e verticale	Solo verticale ^[2]
Sicurezza	Matura, token-based, rate limiting facile	Richiede gestione manuale delle sessioni
Overhead	Un header HTTP per richiesta	Setup iniziale alto, poi minimo

Quando usare cosa

REST API è la scelta giusta per: ^[3]

- Gestione delle regole (aggiunta, modifica, eliminazione)
- Autenticazione e RBAC (meglio token JWT con HTTPS obbligatorio) ^[4]
- Query storiche sui log
- Configurazioni di rete (NAT, zone, policy)

WebSocket è la scelta giusta per: ^[5]

- Stream di log in tempo reale (tipo `tail -f` ma nel browser)
- Dashboard con traffico live (pacchetti/sec, connessioni attive)
- Notifiche di alert critici push dal server
- Stato di salute dei servizi con aggiornamento continuo ^[6]

Alternative da considerare

Oltre a REST + WebSocket, esistono opzioni più specializzate:

- **Server-Sent Events (SSE)**: più semplice di WebSocket per il caso **unidirezionale** (server → client). Perfetto per log stream e alert senza la complessità del full-duplex. Funziona su HTTP/1.1, attraversa più facilmente i proxy — rilevante nel tuo contesto con infrastrutture Proxmox/PRTG.
- **GraphQL + Subscriptions**: usa WebSocket internamente per le subscription ma ti dà flessibilità nel richiedere esattamente i campi che ti servono. Utile se la UI è complessa con molte viste diverse. ^[7]
- **gRPC-Web**: ideale se il backend del firewall espone già API gRPC. Supporta streaming bidirezionale nativo, ottimo per ambienti Kubernetes dove magari gestisci già Envoy o Istio. ^[8]

Architettura raccomandata

Per una firewall UI professionale, il pattern più solido è:

1. **REST API** per tutta la configurazione (con autenticazione JWT, HTTPS forzato, rate limiting) ^[4]
2. **SSE** se il flusso è solo server→client (log, alert) — meno complessità lato server, attraversa i proxy meglio
3. **WebSocket** solo se hai bisogno di interattività bidirezionale in tempo reale (es. terminale integrato, configurazione interattiva live)

Dato il tuo stack (Kubernetes, PRTG, REST API già in uso), un backend con REST + SSE copre il 90% dei casi senza introdurre la complessità di mantenere connessioni WebSocket persistenti attraverso ingress/load balancer.

Stai costruendo la UI sopra un firewall esistente (pfSense, OPNsense, nftables, ecc.) oppure sviluppi il backend da zero?



Sto sviluppando un backend da zero

Perfetto, sviluppare il backend da zero ti dà piena libertà architetturale. Ecco la strategia ottimale per una firewall UI moderna.

Architettura raccomandata: FastAPI + ibrido REST/SSE

Con backend custom, la combinazione più solida è **FastAPI con REST API per la configurazione e SSE per il real-time**. FastAPI supporta entrambi nativamente grazie a Starlette/asyncio, con overhead minimo. ^{[16] [17]}

Layer del Backend

1. REST API — Gestione configurazione

Tutte le operazioni CRUD vanno su endpoint REST classici con JWT auth:

```
# Esempi di endpoint
POST   /api/rules           # Crea nuova regola
GET    /api/rules           # Lista regole attive
PUT    /api/rules/{id}      # Modifica regola
DELETE /api/rules/{id}      # Elimina regola
GET    /api/logs?from=&to=   # Query log storici
POST   /api/zones           # Gestione zone di rete
GET    /api/stats/summary   # Stats aggregate (non real-time)
```

Per l'interfaccia con il sistema, usa `libnftables` via JSON (lo stesso che usa `firewalld` internamente dal 2019) — eviti di chiamare il binario `nft` e hai output strutturato. ^[18]

2. SSE — Monitoring real-time

Server-Sent Events è preferibile a WebSocket per la tua use-case perché il flusso è **unidirezionale** (server → browser) e attraversa meglio proxy e ingress Kubernetes. ^[17]

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio, json

app = FastAPI()

async def log_stream():
    """Tail del log nftables in real-time"""
    while True:
        log_entry = await read_nft_log() # tail journald o file
        yield f"data: {json.dumps(log_entry)}\n\n"
        await asyncio.sleep(0.1)

@app.get("/api/stream/logs")
async def stream_logs():
    return StreamingResponse(
        log_stream(),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"}
    )
```

L'header `X-Accel-Buffering: no` è **fondamentale** se usi Nginx come reverse proxy (rilevante nel tuo contesto infrastrutturale). ^[16]

3. WebSocket — Solo se necessario

Usa WebSocket solo per funzionalità genuinamente bidirezionali in tempo reale: ^[19]

- Terminale integrato nella UI (tipo console nft live)
- Configurazione interattiva con feedback immediato
- Chat/notifiche tra più amministratori connessi

Stack completo consigliato

Layer	Tecnologia	Motivazione
Framework	FastAPI + Uvicorn	Async nativo, SSE/WS built-in, Pydantic per validazione
Auth	JWT + refresh token	Stateless, compatibile con reverse proxy
Firewall engine	python-nftables (libnftables binding)	JSON strutturato, no subprocess
Log streaming	SSE da journald/asyncio	Unidirezionale, proxy-friendly
Storage regole	SQLite o PostgreSQL	Audit trail, rollback configurazione
Task asincroni	FastAPI Background Tasks	Applicazione regole senza bloccare la UI
Frontend	React o Vue + EventSource API	EventSource nativo nel browser per SSE

Pattern importante: CQRS leggero

Per un firewall, separare **scrittura** (applicazione regole → operazione rischiosa, richiede conferma) da **lettura** (log, stats) è una best practice critica. Implementa una coda di operazioni pendenti con approvazione esplicita prima di invocare `libnftables`, così hai un layer di sicurezza anche nell'API stessa. ^[20]

Vuoi che costruisca un prototipo della UI (frontend HTML) o preferisci lo scaffold del backend FastAPI con i pattern sopra descritti?

✱

Io vorrei però utilizzare go per il backend, cose mi puoi dire?

Go è una scelta eccellente per un backend di questo tipo — in realtà è probabilmente superiore a Python/FastAPI per questo use-case specifico, grazie alla concorrenza nativa con goroutines e alla libreria `google/nftables` in pure Go.

Perché Go è ideale per un firewall backend

Go ha caratteristiche strutturalmente adatte a questo dominio: ^[31]

- **Goroutines** per gestire centinaia di connessioni SSE/WebSocket concorrenti con memoria minima
- `google/nftables` è una libreria Go pura che comunica direttamente con il kernel via Netlink (senza chiamare il binario `nft`) ^[32]
- `ngrok/firewall_toolkit` aggiunge un layer di alto livello sopra `google/nftables` per gestire set, regole e filtri eBPF ^[33]
- Compilazione in un singolo binario statico — facile da deployare su Kubernetes o come container minimale

Stack Go consigliato

Layer	Libreria	Note
HTTP Framework	Gin o Fiber	Gin più maturo, Fiber più performante (fasthttp)
WebSocket	gorilla/websocket	Standard de facto in Go ^[34]
SSE	net/http nativo o Gin	Gin supporta SSE con <code>c.SSEvent()</code> ^[35]
nftables	google/nftables + ngrok/firewall_toolkit	Pure Go, no CGO, no subprocess ^[33] ^[31]
Auth	golang-jwt/jwt	JWT con refresh token
Validation	go-playground/validator	Tag-based, integrato con Gin
Config	spf13/viper	YAML/ENV, adatto a K8s ConfigMap

Esempio pratico: SSE con Gin

```
// GET /api/stream/logs
func StreamLogs(c *gin.Context) {
    c.Header("Content-Type", "text/event-stream")
    c.Header("Cache-Control", "no-cache")
    c.Header("X-Accel-Buffering", "no") // fondamentale con Nginx

    clientChan := make(chan string, 10)
    // registra il client nel broker
    broker.Subscribe(clientChan)
    defer broker.Unsubscribe(clientChan)

    c.Stream(func(w io.Writer) bool {
        select {
        case msg := <-clientChan:
            c.SSEvent("log", msg)
```

```

        return true
    case <-c.Request.Context().Done():
        return false // client disconnesso
    }
})
}

```

Il pattern **broker/pub-sub** è fondamentale: un goroutine legge dal kernel/journald e pubblica agli N client SSE connessi, senza creare un goroutine per reader. ^[36]

Interazione con nftables in pure Go

```

import "github.com/google/nftables"

func AddDropRule(sourceIP string) error {
    c, err := nftables.New()
    if err != nil {
        return err
    }

    table := c.AddTable(&nftables.Table{
        Family: nftables.TableFamilyIPv4,
        Name:    "filter",
    })
    chain := c.AddChain(&nftables.Chain{
        Name:    "input",
        Table:   table,
        Type:    nftables.ChainTypeFilter,
        Hooknum: nftables.ChainHookInput,
        Priority: nftables.ChainPriorityFilter,
    })
    // aggiungi regola DROP per IP sorgente
    _ = chain // aggiungi espressioni...
    return c.Flush() // applica atomicamente via Netlink
}

```

`c.Flush()` applica tutte le modifiche **atomicamente** via Netlink, il che è più sicuro del chiamare `nft` da shell. ^[37] ^[31]

Considerazioni architetturali specifiche per Go

Concorrenza e sicurezza: Usa `sync.RWMutex` per proteggere lo stato delle regole condiviso tra goroutines. Per WebSocket, `gorilla/websocket` non è goroutine-safe sulla stessa connessione — usa un goroutine dedicato per la scrittura. ^[38]

Privilege separation: Il binario Go che parla con nftables necessita di `CAP_NET_ADMIN`. In Kubernetes, configura `securityContext.capabilities.add: ["NET_ADMIN"]` nel Pod, evitando di girare come root. ^[38]

Pattern CQRS leggero in Go:

```
[HTTP Handler] → [Command Validator] → [Rule Queue chan] → [nftables Worker goroutine]
                                                    ↓
                                                    [Audit Logger] → [SSE Broker] → [UI]
```

Un singolo goroutine applica le regole sequenzialmente dalla coda, eliminando race conditions sull'interfaccia nftables.

Framework: Gin vs Fiber

Per questo use-case specifico, Gin è la scelta più sicura: ^[39]

- SSE con `c.Stream()` funziona out-of-the-box
- Ecosystem più maturo e documentato
- Fiber usa `fasthttp` che ha limitazioni con il context cancellation di Go standard — rilevante per gestire le disconnessioni client negli SSE ^[40]

Vuoi uno scaffold completo del progetto Go con struttura di directory, Dockerfile e i pattern sopra già implementati?

✱

Stavo pensando a XDP per il motore di firewall base

Ottima scelta architetturale — XDP è esattamente quello che usano Cloudflare, Cilium e Hetzner per i loro data plane ad alte prestazioni. È un salto di paradigma rispetto a nftables, ma le performance giustificano la complessità. ^[46]

XDP: il vantaggio fondamentale

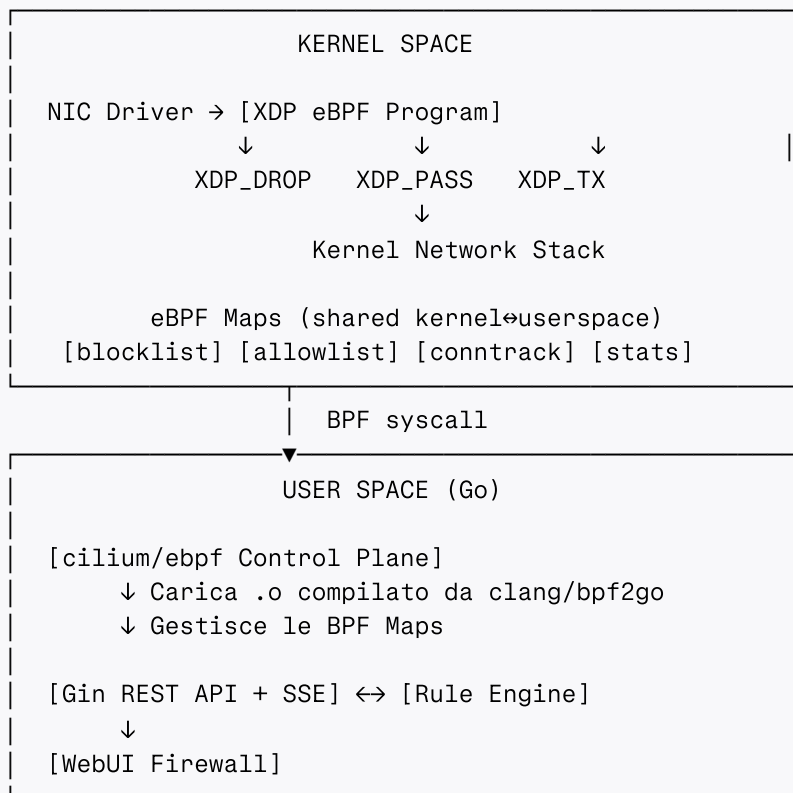
XDP processa i pacchetti a livello di driver NIC, prima che il kernel allochi il socket buffer (SKB) e prima che entri in netfilter. Il gap di performance rispetto a nftables è enorme: ^[47]

Metodo	Pacchetti/sec/core
iptables	~200.000
nftables	~400.000
XDP	5-20 milioni

Un firewall stateful L7 basato su XDP ha raggiunto ~11,8M PPS con ~40% CPU, contro ~2,1M PPS di iptables al 100% di SoftIRQ. ^[48]

Architettura XDP + Go Control Plane

La separazione è netta e ben definita: ^[49] ^[50]



Il control plane Go **non processa pacchetti** — popola le **BPF Maps** (hash map condivise kernel/userspace) che il programma XDP legge in-place ad ogni pacchetto. ^[51]

cilium/ebpf — La libreria Go giusta

cilium/ebpf è la scelta standard, mantenuta da Cloudflare e Cilium, **pure Go senza CGO**. Lo strumento chiave è bpf2go: ^[52]

```
# genera automaticamente il binding Go dal sorgente C
//go:generate go run github.com/cilium/ebpf/cmd/bpf2go \
    -target amd64 \
    -cc clang \
    Firewall ./bpf/firewall.bpf.c -- -I./bpf/headers
```

Questo compila il programma XDP in C e genera automaticamente `firewall_bpf*.go` con tipi Go per caricare il programma e accedere alle map. ^[53]

Struttura del progetto

```
firewall-backend/
├── bpf/
│   ├── firewall.bpf.c      # Programma XDP (C + libbpf)
│   └── headers/            # vmlinux.h, bpf_helpers.h
├── internal/
│   ├── xdp/
│   │   ├── loader.go      # Carica e attacca il prog XDP
│   │   └── maps.go        # CRUD sulle BPF Maps (blocklist, stats)
│   ├── api/
│   │   ├── rules.go       # REST: /api/rules
│   │   └── stream.go      # SSE: /api/stream/logs
│   └── broker/
│       └── broker.go       # Pub-sub per SSE clients
├── cmd/
│   └── main.go
└── go.mod
```

Esempio: aggiornare la blocklist da Go

```
// Aggiunge un IP alla blocklist XDP senza ricaricare il programma
func (m *Maps) BlockIP(ip net.IP) error {
    key := binary.BigEndian.Uint32(ip.To4())
    value := uint8(1)
    // La map è condivisa con il kernel – nessun reload necessario
    return m.BlocklistMap.Put(key, value)
}
```

Il programma XDP nel kernel legge BlocklistMap ad ogni pacchetto e ritorna XDP_DROP istantaneamente se l'IP è presente. La UI chiama il REST API → Go aggiorna la BPF Map → il kernel applica la regola in microsecondi, **senza nessun reload**. ^[51]

Considerazioni importanti

Complessità aggiuntiva rispetto a nftables:

- Il programma XDP va scritto in C con libbpf — Go gestisce solo il control plane ^[54]
- Richiede kernel ≥ 5.2 per BPF CO-RE (Compile Once, Run Everywhere) — rilevante se gestisci nodi con kernel diversi ^[53]
- In Kubernetes servirà CAP_NET_ADMIN + CAP_BPF nel securityContext
- Il conntrack stateful in XDP è complesso: puoi usare XDP_PASS per il traffico non bloccato e delegare il conntrack al kernel, oppure implementarlo in BPF maps (approccio avanzato) ^[48]

Pattern ibrido XDP + nftables: Molte implementazioni production usano XDP per il fast path (DDoS, blocklist IP volumetriche) e nftables per le regole complesse (stateful, L7, NAT). Hetzner ha fatto esattamente questo con hcnets. ^[55] ^[46]

Vuoi lo scaffold completo con bpf2go, il programma XDP di base in C e il control plane Go?



Si, grazie dammi lo scaffold completo.

Ti ho preparato uno scaffold completo per un backend firewall in Go con **control plane Gin + SSE + XDP/eBPF** e packaging base per Docker/Kubernetes. Lo scaffold include `go.mod`, `Makefile`, configurazione `YAML`, `API REST` per block/unblock IP, stream SSE eventi, loader XDP con `cilium/ebpf`, programma `firewall.bpf.c`, `Dockerfile` e manifest Kubernetes di partenza.

Cosa contiene

La struttura è pensata per separare bene API, broker eventi e layer XDP: `cmd/server` per il bootstrap, `internal/api` per routing Gin, `internal/broker` per pub/sub SSE, `internal/xdp` per attach e gestione mappe BPF, `bpf/` per il programma kernel-side e `deployments/` per il deploy iniziale. Il programma XDP usa una hash map `blocked_ips` e una array map `stats`, così puoi aggiornare la blocklist da userspace senza ricaricare il programma eBPF.

Note importanti

Lo scaffold è un'ottima base, ma non è ancora production-ready: il file `vmlinux.h` è volutamente placeholder e va generato dal kernel target con `bpf2go`, mentre `go generate ./...` richiede toolchain eBPF completa disponibile sulla macchina di build. Inoltre, per casi reali conviene estendere questo design con JWT/RBAC, persistenza regole, audit log e soprattutto un pattern ibrido XDP + nftables per stateful filtering, NAT e policy più ricche. [\[61\]](#) [\[62\]](#)

Direzione consigliata

Per il tuo scenario io andrei così: XDP come fast-path per drop immediato di IP/list/set e contatori, poi nftables o un livello successivo per conntrack e regole complesse. Le BPF maps sono il punto giusto di integrazione tra control plane Go e data plane kernel, purché i layout chiave/valore restino coerenti byte-for-byte tra C e userspace. Come stream verso la UI, SSE resta una scelta molto sensata per eventi server→client e richiede header come `X-Accel-Buffering: no` dietro proxy tipo Nginx. [\[63\]](#) [\[62\]](#) [\[64\]](#) [\[61\]](#)

Vuoi che al prossimo passo ti generi anche una **UI HTML di test** per gestire blocklist e vedere gli eventi SSE, oppure preferisci che estenda lo scaffold con **JWT + persistenza PostgreSQL + pattern ibrido XDP/nftables**?



Estendi lo scaffold con JWT + persistenza PostgreSQL + pattern ibrido XDP/nftables. Poi al prossimo passo genera anche una UI HTML di test per gestire blocklist e vedere gli eventi SSE.

Ho esteso lo scaffold con tre blocchi chiave: autenticazione JWT con access token breve e refresh token persistito, persistenza PostgreSQL per utenti/refresh token/audit trail, e pattern ibrido XDP + nftables in cui XDP fa fast-path drop e nftables mantiene il set applicativo lato netfilter. In pratica, una richiesta block-ip ora passa da un service applicativo che aggiorna la mappa XDP, aggiorna il set nftables e registra l'evento su database, così hai sia enforcement rapido sia tracciabilità amministrativa. ^[76] ^[77]

Cosa è stato aggiunto

Lo scaffold ora include moduli auth, db, service e nft, oltre a migration SQL e docker-compose per PostgreSQL locale. Il layer JWT usa access token firmati HS256 e refresh token separati, mentre i refresh token vengono salvati come **hash** nel database, una pratica consigliata per consentire revoca, rotazione e logout reale. ^[78] ^[79]

Scelte architetturali

Per nftables ho seguito il pattern corretto dei set dinamici: invece di creare una regola per ogni IP, il backend inserisce o rimuove elementi da un set IPv4, che poi può essere referenziato da una regola nftables stabile. Questo approccio è coerente con l'uso di google/nftables e con il modello dei set descritto nelle implementazioni Go su nftables. Per l'autenticazione, il modello access+refresh è più adatto di un singolo JWT lungo, perché riduce la finestra di abuso e permette revoca lato server tramite database. ^[80] ^[81] ^[77] ^[79] ^[78] ^[76]

Attenzioni prima della produzione

Questo scaffold è solido come base, ma ci sono alcuni punti da rifinire: i refresh token andrebbero associati a device/sessione, il service block-ip dovrebbe gestire meglio il rollback se XDP riesce e nftables fallisce, e l'API dovrebbe avere RBAC esplicito per ruoli diversi da admin. Inoltre, google/nftables è una libreria pure Go per manipolare nftables, ma in produzione conviene inizializzare table, chain e set in modo idempotente al bootstrap e non solo al primo inserimento. ^[77] ^[82]

Prossimo passo

Al prossimo passaggio posso generarti una **UI HTML di test** con login JWT, form per block/unblock IP, tabella blocklist, pannello stats e stream SSE live, pensata per parlare direttamente con questo scaffold esteso. Questa UI ti sarà utile sia per smoke test rapido sia per validare il flusso completo auth → API → XDP/nftables → SSE.



Si, grazie, generami una UI HTML di test.

Eccoti la UI HTML di test, un singolo file standalone pronto all'uso.

Cosa include la UI

La UI è organizzata in tre sezioni navigate da sidebar:

- **Dashboard** — 4 KPI card animati (pacchetti passati, droppati, IP in blacklist, drop rate), form di blocco rapido IP e pannello degli ultimi eventi SSE live
- **Blocklist** — tabella paginata degli IP bloccati con badge XDP + nft, form di aggiunta con validazione IPv4 e pulsante sblocca per ogni riga
- **Event Stream** — log live SSE a scorrimento con badge per tipo di evento, pulsante reconnect e un **API Tester** integrato per chiamare direttamente `/healthz`, `/api/v1/stats` e `/api/v1/rules` con risposta JSON mostrata inline

Modalità demo automatica

Se il backend Go non è raggiungibile, la UI rileva l'errore di connessione e propone accesso in **modalità demo** con le credenziali `admin / password`. In modalità demo tutte le operazioni (block, unblock, stats animate, eventi SSE simulati) funzionano localmente in memoria senza un backend reale, utile per sviluppare e testare il frontend in isolamento.

Integrazione con il backend

Punterà automaticamente a `http://localhost:8080`, configurabile dall'API Tester.

L'autenticazione usa il flusso JWT completo: login → salva access token in memoria → header `Authorization: Bearer ...` su ogni richiesta protetta. Il refresh token è gestito ma in questa versione di test il rinnovo automatico è lasciato a te per non introdurre logica in background nascosta.

Prossimo passo suggerito

Per il deploy reale ricordati di configurare CORS sul backend Gin (aggiungi `github.com/gin-contrib/cors`) per consentire le richieste dalla UI, e di forzare HTTPS davanti all'EventSource per evitare problemi di mixed content su ambienti di produzione con Nginx.

1. <https://stackoverflow.com/questions/49299362/how-different-between-websocket-and-rest-api>
2. <https://www.wallarm.com/what/websocket-vs-rest-api>
3. <https://dev.to/dashgriva/restapi-vs-websocket-guide-2ljf>
4. <https://pixinvent.com/steps-to-build-a-secure-admin-dashboard-ui-guidelines-f/>
5. <https://www.youtube.com/watch?v=1BWjfi7h3-w>
6. <https://5of10.com/articles/dashboard-design-best-practices>
7. <https://github.com/MasterCloudApps-Projects/REST-gRPC-GraphQL/blob/master/docs/usage/subscriptions.md>
8. <https://www.levo.ai/resources/blogs/grpc-vs-graphql-api-security>

9. <https://www.linkedin.com/pulse/architecting-safer-storefront-why-we-built-waf-pro-jaber-kibria-ircmc>
10. <https://www.lazarev.agency/articles/dashboard-ux-design>
11. <https://github.com/ysugimoto/grpc-graphql-gateway/blob/master/README.md>
12. <https://grafbase.com/blog/extend-rest-and-grpc-apis-with-graphql-using-grafbase>
13. <https://www.resolution.de/post/dashboard-design-best-practices/>
14. <https://wundergraph.com/blog/is-grpc-really-better-for-microservices-than-graphql>
15. <https://www.geeksforgeeks.org/computer-networks/difference-between-rest-api-and-web-socket-api/>
16. <https://github.com/MatthewScholefield/fastapi-sse>
17. <https://dev.to/zachary62/build-an-llm-web-app-in-python-from-scratch-part-4-fastapi-background-tasks-sse-21g4>
18. <https://firewalld.org/2019/09/libnftables-JSON>
19. <https://www.youtube.com/watch?v=uCCEasljr74>
20. <https://dev.to/padie78/real-world-backend-architecture-patterns-you-should-know-17kd>
21. <https://v0.app/chat/ai-driven-waf-system-XonosyXDKZa>
22. <https://github.com/parag290/Firewall-for-Linux-using-Python>
23. https://github.com/atulkp018/Application_firewall_frontend
24. <https://github.com/zhiyuan8/FastAPI-websocket-tutorial>
25. <https://systemdr.systemdrcd.com/p/iot-backend-system-design-patterns>
26. <https://en.wikipedia.org/wiki/Nftables>
27. https://brennan.io/papers/pywall_report.pdf
28. <https://www.peakhour.io/learning/threat-detection/implementing-real-time-threat-monitoring/>
29. <https://github.com/AndreyKlychnikov/fastapi-sse>
30. <https://firewalld.org/2018/07/nftables-backend>
31. https://tomdnetto.net/post/linux_networking_from_go_nftables
32. <https://packages.debian.org/unstable/golang-github-google-nftables-dev>
33. https://github.com/ngrok/firewall_toolkit
34. <https://centrifugal.dev/blog/2020/11/12/scaling-websocket>
35. <https://gist.github.com/SubCoder1/3a700149b2e7bb179a9123c6283030ff>
36. <https://dev.to/rafaelgfirmينو/golang-and-sse-3l56>
37. https://tomdnetto.net/post/advanced_nftables_with_go
38. <https://www.superdevacademy.com/en/blogs/go-enterprise-websocket-security-guide-best-practices>
39. <https://docs.gofiber.io/recipes/sse/>
40. <https://www.youtube.com/watch?v=UmAnbKWDNK8>
41. <https://pkg.go.dev/github.com/wallarm/api-firewall>
42. <https://blog.csdn.net/JineD/article/details/130883094>
43. <https://www.youtube.com/watch?v=pKpKv9MKN-E>
44. https://dev.to/jones_charles_ad50858dbc0/go-websocket-programming-build-real-time-apps-with-ease-1o57
45. <https://github.com/topics/nftables?l=go&o=desc&s=forks>

46. https://www.youtube.com/watch?v=3G_gL7nvwVE
47. <https://thelinuxnotes.com/ebpf-and-xdp-ultra-fast-packet-processing-and-ddos-protection-in-linux/>
48. <https://dev.to/nevinshine/pushing-xdp-limits-how-i-built-a-stateful-l7-firewall-in-ebpf-3ej6>
49. https://www.usenix.org/system/files/login/articles/login_spring18_05_choudhury.pdf
50. <https://lpc.events/event/2/contributions/71/attachments/17/9/presentation-lpc2018-xdp-tutorial.pdf>
51. <https://dev.to/aaravjoshi/high-performance-packet-processing-combining-ebpf-xdp-and-go-for-million-packet-per-second-28i>
52. <https://github.com/cilium/ebpf>
53. <https://www.inspektor-gadget.io/blog/2022/09/rewriting-the-control-plane-of-bcc-tools-in-golang/>
54. <https://people.netfilter.org/hawk/presentations/ebplane2019/xdp-bpf-building-blocks.pdf>
55. <https://dev.to/arzenlabscom/building-a-high-performance-ddos-mitigation-pipeline-with-nftables-and-xdp-2e0j>
56. [https://lpc.events/event/11/contributions/937/attachments/868/1660/Plumbers 2021 cilium ebpf.pdf](https://lpc.events/event/11/contributions/937/attachments/868/1660/Plumbers%2021%20cilium%20ebpf.pdf)
57. https://docs.redhat.com/de/documentation/red_hat_enterprise_linux/9/epub/configuring_firewalls_and_packet_filters/using-xdp-filter-for-high-performance-traffic-filtering-to-prevent-ddos-attacks_firewall-packet-filters
58. https://www.linkedin.com/posts/giuseppe-ognibene-538930148_github-pinoogniebpfsamples-this-repo-activity-7257508451570900992-5ZM7
59. <https://pkg.go.dev/github.com/cilium/ebpf/examples@v0.0.0-20220819183058-4420605496c5/xdp>
60. <https://press.um.si/index.php/ump/catalog/view/735/1018/2689>
61. <https://docs.ebpf.io/linux/concepts/maps/>
62. <https://stackoverflow.com/questions/72191414/thread-safe-operations-on-xdp>
63. <https://oneuptime.com/blog/post/2026-01-27-sse-different-frameworks/view>
64. <https://sund.site/posts/2023/go-server-side-events/>
65. <https://kkovacs.eu/resilient-server-sent-events-sse-with-golang/>
66. <https://github.com/cilium/ebpf/blob/main/docs/ebpf/guides/getting-started.md>
67. <https://github.com/xdp-project/xdp-tutorial/blob/main/basic03-map-counter/README.org>
68. <https://github.com/cilium/ebpf/blob/main/cmd/bpf2go/main.go>
69. <https://dev.to/zhufuyi/is-real-time-pushing-in-go-too-hard-try-sponge-sse-and-get-it-done-in-one-click-3c94>
70. <https://mcorbin.fr/pages/xdp-introduction/>
71. <https://oneuptime.com/blog/post/2026-01-07-ebpf-go-cilium-ebpf/view>
72. <https://github.com/gin-gonic/gin/issues/3658>
73. <https://jvns.ca/blog/2017/04/07/xdp-bpf-tutorial/>
74. <https://pkg.go.dev/github.com/cilium/ebpf>
75. <https://github.com/cilium/ebpf>
76. https://tomdnetto.net/post/advanced_nftables_with_go
77. <https://pkg.go.dev/github.com/google/nftables>
78. <https://stackoverflow.com/questions/68385445/how-to-correctly-store-jwt-refresh-tokens-in-a-database>
79. <https://blog.intertoons.com/building-secure-apis-with-jwt-access-and-refresh-tokens-9988eb6414ce>
80. <https://github.com/sbezverk/nftableslib>

81. <https://developer.vonage.com/en/blog/using-jwt-for-authentication-in-a-golang-application-dr>
82. <https://github.com/google/nftables>
83. <https://github.com/Bingmang/gin-restful-best-practice>
84. <https://takx.xyz/go-gin-auth-session>
85. <https://github.com/RakibSiddiquee/golang-gin-jwt-auth-crud>
86. <https://pkg.go.dev/github.com/appleboy/gin-jwt/v2>
87. <https://blog.stackademic.com/golang-authentication-with-gin-and-gorm-c87d7cba1ed3>
88. <https://stackoverflow.com/questions/77613007/troubleshooting-nftables-table-creation-with-go>
89. <https://medium.com/@patelpriysnshu2410/from-zero-to-production-deploying-a-secure-go-api-with-gin-gorm-and-jwt-auth-2025-guide-6aa733d68ff7>
90. https://medium.com/@brian_njuguna/jwt-authentication-and-refresh-token-implementation-with-nodejs-express-postgres-and-drizzle-orm-d6709ca755e1